Multithreading is a powerful feature of Python that allows developers to create applications that can execute multiple things simultaneously. This can lead to improved performance and increased efficiency in applications that perform complex tasks. In this tutorial, we will cover the basics of multithreading in Python and how to use it to improve the performance of your applications.

Why Use Multithreading?

Suppose you have a large dataset that needs to be processed, such as performing complex calculations or analyzing data. Processing this data sequentially could take a long time and result in a slow and inefficient process.

By using multithreading, you can split the data processing task into smaller chunks and process them concurrently using separate threads. This can greatly improve the processing speed and efficiency of the application, allowing you to analyze and process large datasets more quickly (this is actually not entirely accurate in the case of Python, which we will discuss in the future series — but this is a good example to mentally visualize why one might need to do tasks in parallel).

For example, suppose you are working with a dataset that contains millions of rows of data. By using multithreading, you can split the dataset into smaller chunks and process them concurrently using separate threads. This can result in a faster and more efficient data processing process, allowing you to analyze and extract insights from the data more quickly and effectively.

Multithreading can be used in a variety of scenarios to improve the performance of applications. Another example can be a program that needs to perform a long-running task, such as downloading a large file from the internet. Without multithreading, the program would have to wait for the download to complete before continuing with other tasks. By using multithreading, the program can continue to perform other tasks while the download is in progress, leading to improved performance and reduced wait times.

What is Multithreading?

Multithreading is a programming technique that allows multiple threads to run concurrently within the same program. A thread is a lightweight process that can perform a specific task while the main program continues to run. By using

multithreading, a program can make use of the available processing power of the system to perform multiple tasks simultaneously.

Creating Threads in Python

To create a thread in Python, you need to import the threading module and create an instance of the Thread class. The Thread class takes a target function as an argument, which is the function that will be executed in the thread:

```
import threading


def my_function():
    print('Hello from a thread!')


my_thread = threading.Thread(target=my_function)
my_thread.start()
```

In this example, we import the threading module and define a function called `my_function()`. We then create an instance of the Thread class and pass `my_function()` as the target function. Finally, we start the thread by calling the `start()` method.

When you run this code, you should see the following output:

```
Hello from a thread!
```

This output is generated by the thread that was created by the `my_thread.start()` method. Note that the main program continues to run after the thread is started.

Passing Arguments to a Thread

You can pass arguments to a thread by specifying them as additional arguments when creating the Thread instance:

```
import threading


def print_text(name):
    print('Hello from', name)
```

```
my_thread = threading.Thread(target=print_text, args=('Alice',))
my_thread.start()
```

In this example, we define a function called `print_text()` that takes a single argument `name`. We then create an instance of the Thread class and pass `print_text()` as the target function, along with the argument `'Alice'`. Finally, we start the thread by calling the `start()` method. When you run this code, you should see the following output:

```
Hello from Alice
```

Starting Multiple Threads

We can also start multiple threads and execute different blocks of code at the same time. This way, the program can perform multiple operations without blocking the others. Let's enhance the example above and call the `print_text` function multiple times in different threads:

```
import random
import threading
from time import sleep


def print_text(text):
    sleep_seconds = random.uniform(0, 1)
    sleep(sleep_seconds)
    print('Hello from', text)


names = ['Alice', 'Bob', 'Charlie', 'Dave', 'Anna', 'Mary', 'John', 'David']
threads = []
for name in names:
    thread = threading.Thread(target=print_text, args=(name,))
    print('Starting thread', name)
    thread.start()
    threads.append(thread)

for name, thread in zip(names, threads):
    print('Joining thread', name)
    thread.join()

print('Done!')
```

Notice the `sleep(sleep_seconds)` which acts as a way of simulating different computations which might take different times on different threads in other scenarios. This program can print different outputs as the order is not guaranteed. On output can be the following:

```
Starting thread Alice
Starting thread Bob
Starting thread Charlie
Starting thread Dave
Starting thread Anna
Starting thread Mary
Starting thread John
Starting thread David
Joining thread Alice
Hello from Anna
Hello from David
Hello from Dave
Hello from Mary
Hello from Alice
Joining thread Bob
Hello from Bob
Joining thread Charlie
Hello from Charlie
Joining thread Dave
Joining thread Anna
Joining thread Mary
Joining thread John
Hello from John
Joining thread David
Done!
```

To make sure all the threads have finished their execution before stopping the program, we can use the `join()` method on each thread which would make sure the main program waits for that thread to finish before moving on.

Finally, in our example, the last line of code prints the text `Done!` after which the whole program stops the execution.

Race Conditions in Multithreading

Race conditions occur in a multithreaded environment when two or more threads access a shared resource (such as a variable, file, or network connection) concurrently and attempt to modify it at the same time. When this happens, the

behavior of the program becomes unpredictable and can result in unexpected outcomes or errors.

For example, imagine you have two threads that are both trying to increment a counter variable at the same time. The code might look something like this:

```
import threading


counter = 0

def increment():
    global counter
    counter += 1


thread1 = threading.Thread(target=increment)
thread2 = threading.Thread(target=increment)

thread1.start()
thread2.start()

thread1.join()
thread2.join()

print(counter)
# The output is unpredictable
# This can print either 1 or 2
```

In this example, we define a global variable `counter` and a function called `increment()` that increments the counter variable. We then create two threads that call the `increment()` function and start them using the `start()` method. Finally, we use the `join()` method to wait for the threads to finish before printing the value of the `counter` variable.

The problem with this code is that both threads are modifying the `counter` variable at the same time, which can result in a race condition. Depending on how the operating system schedules the threads, the output of the program could be unpredictable. For example, the output could be 1 (if one thread finishes before the other), 2 (if both threads finish without interfering with each other), or some other value (if the threads interfere with each other).

In reality, the code above will perform fine in most cases. To “force” Python result in a race condition, we need to “allow” the interpreter to context-switch halfway through the code, which will switch between different threads causing inconsistencies:

```
import threading
from time import sleep

counter = 0


def add(n):
    """ Add n to the global counter with a loop """
    global counter
    for _ in range(n):
        res = counter   # Copy the current value of counter
        sleep(0)        # Context switch
        res += 1        # Increment the copy
        sleep(0)        # Context switch
        counter = res   # Write the copy back to counter


# Create 100k threads that each add 100 to the counter
threads = []
for i in range(100_000):
    thread = threading.Thread(target=add, args=(100,))
    thread.start()
    threads.append(thread)

for thread in threads:
    thread.join()

print(counter, 'Should be:', 100_000 * 100)
# 396584 Should be: 10000000
```

To avoid race conditions, you need to use synchronization mechanisms such as locks, semaphores, or barriers to ensure that only one thread can access a shared resource at a time. By using these mechanisms, you can prevent race conditions and ensure that your program behaves predictably and correctly in a multithreaded environment.

We will discuss locks, semaphores, and all the synchronization methods in the next post.